

# Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks  
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

| Student & Hardware Metadata Field          | Laboratory Session Specifications & Target Machine Declaration |
|--------------------------------------------|----------------------------------------------------------------|
| Student Full Name                          | Adel Kunuspekova                                               |
| Student Identification Number              | 230103231                                                      |
| Date & Laboratory Session                  | 17.09.2026 / 02N                                               |
| Workstation / Laptop Make & Model          | 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz                 |
| Processor SKU / SoC Model                  | e.g., AMD Ryzen 7 5800H / Apple M2 Pro / Intel Core i7-12700H  |
| Core Topology (Physical P-cores / E-cores) | Physical Cores: 4   Logical SMT Threads: 8                     |
| Cache Hierarchy Capacity                   | L1: 320 KB   L2: 5 MB   L3: 8 MB                               |
| Host OS & Python Runtime                   | e.g., Linux Kernel 6.8 / Ubuntu 24.04 LTS / CPython 3.11.8     |

## MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results\_<student\_id>.txt' before the 120-minute timer elapses.

## TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [ (1 - P) + (P / p) ]$$

where  $S(p)$  is the speedup ratio  $T(1)/T(p)$ ,  $P$  is the strictly parallelizable fraction of the algorithm, and  $p$  is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

## EXECUTION PROTOCOL

Create a script named 'task1\_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1\_amdahl.py'. Record the single-process baseline  $T(1)$  and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

| Worker Cores (p) | Measured Time T(p) [s] | Observed Speedup S(p) | Ideal Linear Speedup | Parallel Efficiency [S(p)/p * 100] |
|------------------|------------------------|-----------------------|----------------------|------------------------------------|
| p = 1 (Baseline) | 3.3835                 | 1.00x                 | 1.00x                | 100.0%                             |
| p = 2            | 2.1545                 | 2.02x                 | 2.00x                | 101                                |
| p = 4            | 1.9447                 | 2.23x                 | 4.00x                | 55,75                              |
| p = 8            | 2.5160                 | 1.73x                 | 8.00x                | 21,625                             |
| p = 12           | 2.2793                 | 1.91x                 | 12.00x               | 15,917                             |
| p = 16           | 3.5620                 | 1.22x                 | 16.00x               | 7,625                              |

**[Q1.1]** Identify the exact inflection point (core count  $p^*$ ) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

The speedup curve peaks at  $p^* = 4$ , exactly matching my CPU's 4 physical cores (Intel i5-1135G7, Tiger Lake, no E-cores). From  $p=1 \rightarrow 4$ , speedup rises steadily ( $1.00x \rightarrow 1.74x$ ) as each process gets a dedicated physical core. Past  $p=4$ , performance degrades ( $p=8$ :  $1.35x$ ) because processes now compete for the 4 physical cores via SMT — logical threads 5–8 are hyperthreads sharing ALU/FPU and cache with their physical sibling, which barely helps this modulo-heavy integer workload. By  $p=16$ , oversubscription (16 processes on 8 logical slots) plus fork/context-switch overhead drives speedup below baseline ( $0.95x$ ).

**[Q1.2]** Using your empirical measurements for  $T(1)$  and  $T(4)$ , calculate the exact value of  $P$  (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation:  $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$ . (9 Points)

*Student Response & Architectural Justification:*

$S(4) = T(1)/T(4) = 3.3835/1.9447 = 1.7399x$   
Solving  $1.7399 = 1/[(1-P) + P/4]$ :  
 $(1-P)+P/4 = 0.5748 \rightarrow 1 - 0.75P = 0.5748 \rightarrow P = 0.4252/0.75$   
 $P \approx 0.567$

**[Q1.3]** Compute the Asymptotic Speedup Limit  $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$  using your calculated  $P$  from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

*Student Response & Architectural Justification:*

$S_{\max} = 1/(1-P) = 1/0.4331 \approx 2.31x$   
Even with a 128-core server, this workload could never exceed  $\sim 2.3x$  speedup — the  $\sim 43\%$  serial fraction (process spawn/teardown, IPC overhead) becomes the hard ceiling; cores beyond  $\sim 4$  would sit largely idle for this specific code and problem size.

## TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

*Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation*

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

### ■ EXECUTION PROTOCOL

Create 'task2\_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements ( $16 * 8$  bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

| Memory Layout Configuration             | Trial 1 [s] | Trial 2 [s] | Trial 3 [s] | Median Time [s] | Slowdown Penalty Multiplier |
|-----------------------------------------|-------------|-------------|-------------|-----------------|-----------------------------|
| Adjacent Indices (False Sharing Active) | 1.5464      | 1.1343      | 1.0249      | 1.2352          | — [Baseline]                |
| Padded Indices (Cache-Line Aligned)     | 1.1401      | 1.1163      | 1.0835      | 1.1133          |                             |

**[Q2.1]** Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

*Student Response & Architectural Justification:*

In CPython, a list stores an array of 8-byte (64-bit) pointers to PyObject instances. A 64-byte cache line therefore holds  $64/8 = 8$  pointer slots. My stride of 16 elements =  $16 \times 8 \text{ bytes} = 128 \text{ bytes}$  between adjacent threads' indices. Since  $128 \text{ bytes} > 64 \text{ bytes}$  (a full cache line width), each thread's target index is guaranteed to fall on a separate cache line — in fact one full empty line apart — eliminating any possibility of two threads sharing a line.

**[Q2.2]** Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

*Student Response & Architectural Justification:*

Initially the cache line holding indices 0 and 1 is Shared (or Invalid if untouched) in both cores' caches. Core 0 writes index 0 → its cache controller broadcasts an invalidation, the line becomes Modified in Core 0's cache and Invalid in Core 1's. When Core 1 then writes index 1 (same line), it must first fetch the now-Modified line from Core 0 (write-back to memory or cache-to-cache transfer), invalidating Core 0's copy — the line becomes Modified in Core 1, Invalid in Core 0. This ping-pongs on every single increment from either core, even though 0 and 1 are logically independent variables — pure false sharing.

**[Q2.3]** In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

*Student Response & Architectural Justification:*

Two standard mechanisms: (1) C++11 alignas(64) forces a variable/struct to start on a 64-byte boundary, guaranteeing it occupies its own cache line without manual array offset tricks; (2) Java's @sun.misc.Contended annotation (or Go's manual struct field padding with unused byte arrays) inserts padding bytes around a field at the JVM/compiler level so independently-accessed fields never share a line — achieving the same effect as my index-16 stride, but automatically and portably rather than by hand-tuning array offsets.

### TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

*Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning*

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse-grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

#### ■ EXECUTION PROTOCOL

Create 'task3\_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

| Counter Architecture                | Target Sum | Actual Empirical Sum | Data Corruption Defect Count | Execution Time [s] | Contention Overhead |
|-------------------------------------|------------|----------------------|------------------------------|--------------------|---------------------|
| Unsafe Accumulator (Race Condition) | 2,000,000  | 2000000              | 0                            | 0.1492             | 1.00x<br>[Baseline] |
| Locked Accumulator (threading.Lock) | 2,000,000  | 2,000,000            | 0 (Protected)                | 0.4954             | 3.32                |

**[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read-modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)**

*Student Response & Architectural Justification:*

My empirical result was 2,000,000 / 2,000,000 — zero corruption, despite `val += 1` being a non-atomic read-modify-write (LOAD `val` → ADD 1 → STORE `val`) at the bytecode level. This is because CPython's GIL only allows one thread to execute bytecode at a time, and by default only switches threads every ~5ms (`sys.getswitchinterval()`) or after ~100 bytecode instructions — so a thread's LOAD/ADD/STORE sequence for one increment almost always completes atomically before a switch occurs. Corruption can still happen if a context switch lands exactly between LOAD and STORE, but at this op count it's statistically rare rather than impossible — a different run isn't guaranteed to also show zero defects.

### ■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x.

Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

**[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)**

*Student Response & Architectural Justification:*

```
def bench_lockless():
    partials = [0] * NUM_THREADS
    ops_per_thread = TOTAL_OPS // NUM_THREADS

    def work(idx):
        local = 0
        for _ in range(ops_per_thread):
            local += 1
        partials[idx] = local

    threads = [threading.Thread(target=work, args=(i,)) for i in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return sum(partials), time.perf_counter() - start
```

**[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)**

*Student Response & Architectural Justification:*

Locks (even adaptive spinlocks/RW-locks) still force serialization of the critical section and pay recurring costs: kernel futex calls, cache-line ping-ponging on the lock variable itself, and thread suspension/wakeup. Thread-local accumulation removes the shared mutable state entirely, so there's nothing to contend over during the hot loop — each core works purely in its own cache lines with zero cross-core coordination, paying a synchronization cost only once, at the final reduction. This is the general principle "the fastest lock is no lock": eliminating contention structurally scales better than making contention faster to resolve.

## TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

*Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation*

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple

cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

## ■ EXECUTION PROTOCOL

Create 'task4\_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```
import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()
```

| Worker Allocation       | Compute-Bound Time [s] | Compute Scaling [Tw / T1] | Memory-Bound Time [s] | Memory Degradation [Tw / T1] |
|-------------------------|------------------------|---------------------------|-----------------------|------------------------------|
| w = 1 Worker (Baseline) |                        | 1.00x                     |                       | 1.00x                        |
| w = 2 Workers           |                        |                           |                       |                              |

| Worker Allocation | Compute-Bound Time [s] | Compute Scaling [ $T_w / T_1$ ] | Memory-Bound Time [s] | Memory Degradation [ $T_w / T_1$ ] |
|-------------------|------------------------|---------------------------------|-----------------------|------------------------------------|
| w = 4 Workers     |                        |                                 |                       |                                    |

**[Q4.1]** Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

**[Q4.2]** Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

**[Q4.3]** Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

| Assessment Category       | Evaluation Criteria & Standards                                                                                                             | Max Points | Score Awarded |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|------------|---------------|
| Task 1: Amdahl Scaling    | Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.    | 25         |               |
| Task 2: Cache Coherence   | Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.          | 25         |               |
| Task 3: Sync & Lockless   | Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.    | 25         |               |
| Task 4: Roofline Analysis | Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification. | 25         |               |
| TOTAL EVALUATION          | Comprehensive Silicon Profiling & Architectural Systems Mastery                                                                             | 100        |               |

|                                                                                                                                                                  |                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Instructor / TA Walk-Around Verification Check:                                                                                                                  | Laboratory Submission Audit Timestamp:                |
| <div>[ ] Task 3 Unsafe Corruption Verified</div> <div>[ ] Task 2 Cache Slowdown Verified</div> <div>[ ] Terminal Logfile 'results_&lt;id&gt;.txt' Uploaded</div> | <div>Examiner Signature: _____</div> <div>Date:</div> |

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

| Operating System            | Command to Retrieve CPU SKU & Core Topology                                                                               | Command to Inspect Cache & Memory Bus                                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Linux (Ubuntu/Debian)       | lscpu<br>grep -E 'model name cpu cores' /proc/cpuinfo                                                                     | lscpu -C<br>sudo dmidecode --type memory   grep Speed                                 |
| macOS (Apple Silicon/Intel) | sysctl -n machdep.cpu.brand_string<br>sysctl hw.perflevel0.physicalcpu                                                    | sysctl hw.l1d cachesize hw.l2 cachesize<br>system_profiler SPMemoryDataType           |
| Windows (PowerShell / CMD)  | Get-CimInstance Win32_Processor   Select Name, NumberOfCores<br>wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors | Get-CimInstance Win32_CacheMemory<br>wmic memorychip get speed,capacity,devicelocator |

## Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

```
import time
```

```
from multiprocessing import Pool
```

```
import threading
```

```
def cpu_work(n):
```

```
    # Pure CPU crunch
```

```
    count = 0
```

```
    for i in range(n):
```

```
        count += i * i
```

```
    return count
```

```
# Run both experiments with total_work = 50_000_000 split across T workers
```

```
# Test T in [1, 2, 4, 8, 16, 32]
```

### Deliverables:

1. **The Empirical Curve:** Plot  $T$  (workers) on the X-axis vs. Execution Time on the Y-axis for both `threading.Thread` and `multiprocessing.Pool`.
2. **The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
3. **The Architectural Autopsy:**
  - o Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
  - o Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*?
  - o Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when  $T > \text{physical cores}$  using context-switching overhead.